Brickts SQL course — Week 1 — First SQL questions

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 1 — First SQL questions

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 01: Day 01 — Open the SQL workbench
- Day 02: Day 02 — Read tables and name the row

- Day 03: Day 03 — Filter rows with explicit logic
- Day 04: Day 04 — Order results and take a bounded look
- Day 05: Day 05 — Count missing values before averaging
- Day 06: Day 06 — Group samples into accountable summaries
- Day 07 (weekly challenge): Day 07 — Week 1 challenge — the first site report

---

Day 01 — Open the SQL workbench

Week 1 · 45–75 minutes. Prerequisites: none; start here.

Goal

Start the disposable course database, connect with `psql`, and explain the difference between a terminal command, a SQL statement, and a database server. Your first successful query should be something you understand completely.

Before you start

Choose the local lab or follow the remote Timescale setup on a separate test machine. Read the prerequisites and database isolation rules. The commands below are for the local route; the remote guide shows where each command runs. From the repository root, run these terminal commands separately:

```
uv run python lessons/sql/lab.py up
uv run python lessons/sql/lab.py load
uv run python lessons/sql/lab.py psql
```

Use only the dedicated `brickts_course` database. Save your notes and SQL under your own `student-work/sql/day01/` directory. Keep passwords and connection strings out of evidence files.

Concept

The terminal runs programs. `psql` is a client program: it sends SQL to PostgreSQL, the server that stores tables and executes queries. Closing the client does not delete the database. A SQL statement normally ends with a semicolon; pressing Enter alone can leave the client waiting for the rest of it.

A literal is a value written directly in a statement. Single quotes surround SQL text values. An alias gives an output column a useful name; it does not create a stored column. Double quotes have a different job: they quote identifiers. We will use simple lowercase identifiers so that distinction stays visible.

Tiny example

Inside `psql`, run:

```
SELECT 'synthetic building lab' AS
         purpose,
       2 + 3 AS arithmetic_result;
SELECT current_database() AS
         database_name;
```

The first query returns one row with arithmetic result `5`; the second should name `brickts_course`. Neither query reads building samples. Try `\conninfo` to inspect the current connection and `\q` to leave the client. Backslash commands belong to `psql`, so they do not need SQL semicolons.

SQL challenge

Reconnect and write a query that returns your chosen report title, the number of minutes in an hour, and a clearly named numeric

expression. Use separate aliases that describe each result. Record `SELECT version();` so your notes identify the actual server. Explain which process produced the answer and which process displayed it.

Experiment

Enter `SELECT 10 +` and press Enter. Observe the changed prompt, then finish with `5;`. Repeat with an accidental unfinished statement and use Ctrl+C to cancel it. Do not paste terminal commands into the SQL prompt. Predict what each input will do before running it.

Acceptance checks

- The connection identifies `brickts_course`, and your arithmetic has the expected result.
- You can cancel unfinished input and successfully run another statement.
- Your notes distinguish the shell, `psql`, SQL, and PostgreSQL in your own words.

Stretch and reflection

Use `\?` and `\h SELECT` to find client and SQL help. Which information belongs to the client, and which describes the server language?

Record one error message and the correction you understood. Read week 1 references before changing installation settings.

Day 02 — Read tables and name the row

Previous: Day 1 · **Week 1 · 45–75 minutes.**
Prerequisites: days 1–1, or equivalent skills.

Goal

Read metadata tables with `SELECT`, choose useful columns, and state what one returned row represents. This is the first step toward trusting a report.

Before you start

Connect using `uv run python lessons/sql/lab.py psql`. Keep the lab guide available. The fixture has two synthetic sites, three AHUs, and fourteen measured roles per AHU. An AHU is an air handling unit; no HVAC expertise is required for today's inventory queries.

Concept

A table contains rows described by named columns. A query selects columns from an input relation. Its result is another relation,

not necessarily a stored table. The grain is the meaning of one row: `course.site` has one row per site, `course.equipment` one per equipment identity, and `course.point` one per measured point identity.

The schema prefix in `course.site` tells PostgreSQL which namespace contains the table. Use qualified names throughout the course so a similarly named table elsewhere cannot change your query accidentally. A UUID is an opaque identity value. Copy it when needed; do not reconstruct it from a friendly label. North and south both contain an AHU labeled `Supply A`.

Tiny example

```
SELECT site_id, label, tz
FROM course.site;

SELECT equipment_id, site_id, label, kind
FROM course.equipment;
```

The first result has two rows, including site IDs `SYN-NORTH` and `SYN-SOUTH`. The second has three rows. Row order is unspecified until you ask for an order; an apparent order in one run is not a promise.

In `psql`, `\d course.point` shows column definitions without reading sample values. `SELECT *` is useful for a quick inspection, but an explicit report should list its columns so its shape remains deliberate.

SQL challenge

Create three inventory statements: site IDs and labels; equipment identities, labels, and site IDs; point identities, roles, and units. Add understandable output aliases to at least one statement. Write a sentence describing each result's grain. Check the point inventory has forty-two rows, with roles such as `duct_static` in Pa and `kw` in kW.

Experiment

Select only `label` from `course.equipment`. Then select `equipment_id, site_id, label`. Compare what a reader can identify in each result. Why can two matching labels refer to different equipment? Try swapping selected column order; explain why this changes presentation without changing equipment identities.

Acceptance checks

- Your inventories contain 2 site rows, 3 equipment rows, and 42 point rows.

- You can identify the same equipment even when its label is hidden.
- Every saved query names its source schema and has a stated row meaning.

Stretch and reflection

Inspect `course.sample` with `\d`, without printing its entire contents. Its row represents a point at an instant within a site. Which columns appear necessary to distinguish two measurements? Record your hypothesis; keys are studied next week. Consult week 1 references for `SELECT` and `psql` details.

Previous: Day 1 ·

Day 03 — Filter rows with explicit logic

Previous: Day 2 · **Week 1 · 45–75 minutes.**
Prerequisites: days 1–2, or equivalent skills.

Goal

Use `WHERE`, comparisons, `AND`, `OR`, and parentheses to request a precise subset of point metadata. Translate an English requirement into a condition you can explain.

Before you start

Open the course `psql` client. Revisit yesterday's distinction between point identity and equipment label. Today's queries inspect metadata only; a point's role describes what it measures, while its unit describes the scale of its numeric values.

Concept

`WHERE` keeps rows for which its condition is true. Text comparisons use single-quoted values: `site_id = 'SYN-NORTH'`. The expression `role = 'sat'` compares the role column to a

text literal. Writing a column name inside single quotes would instead create a constant string.

`AND` requires both conditions; `OR` accepts either. PostgreSQL evaluates `AND` before `OR`. Parentheses make your intended grouping visible, including to your future self. A logically valid query can still answer the wrong question if its grouping is wrong. `IN` is a compact way to test membership in a short set of values.

Tiny example

```
SELECT point_id, site_id, role, unit
FROM course.point
WHERE site_id = 'SYN-NORTH'
  AND role IN ('duct_static',
               'duct_static_sp');
```

Expect four rows: two pressure roles for each of the two northern AHUs. Every returned unit is `Pa`. The query does not select equipment by its human label and does not read sample histories.

SQL challenge

Write a query for northern points whose role is either `sat` or `sat_sp`, showing point identity, equipment identity, role, and unit. Write

another for all southern points. Before running them, predict the row counts from the fixture inventory. Then write the first requirement a second way using `OR` with explicit parentheses.

Describe why the two versions express the same membership condition. Treat your expected counts as checks, not values to hardcode into the query.

Experiment

Compare these predicates in otherwise identical queries:

```
WHERE site_id = 'SYN-NORTH' AND role =
          'sat' OR role = 'sat_sp'
```

```
WHERE site_id = 'SYN-NORTH' AND (role =
          'sat' OR role = 'sat_sp')
```

The first returns five rows, including the southern SAT setpoint; the second returns four. Find the extra row by inspecting its site and role. Explain the result using the grouping rules rather than the order of the written words.

Acceptance checks

- The northern SAT pair returns 4 rows; the southern inventory returns 14.
- The membership and parenthesized `OR` versions select identical point identities.
- Your explanation accounts for the fifth row in the deliberately ambiguous example.

Stretch and reflection

Filter roles with `unit = 'degC'` and inspect the result. Does knowing a unit tell you which physical quantity a point represents? Explain why SAT, outdoor temperature, and zone temperature need distinct roles despite sharing a unit. See week 1 references for Boolean expressions and comparisons.

Previous: Day 2 ·

Day 04 — Order results and take a bounded look

Previous: Day 3 · **Week 1 · 45–75 minutes.**
Prerequisites: days 1–3, or equivalent skills.

Goal

Create predictable report orderings and inspect a small sample of a large table without confusing the displayed rows with the entire dataset.

Before you start

Connect to the course database and review the grain of `course.sample`: one row per site, point identity, and instant. The long table stores many roles in the same `value` column. Units live with point metadata; a naked value is not yet an interpretable engineering measurement.

Concept

`ORDER BY` requests an ordering. `ASC` means ascending and `DESC` descending. Multiple expressions break ties from left to right. If

your final ordering expressions still tie, the order within those ties is unspecified. `LIMIT` bounds returned rows after sorting, and it does not change what exists in the table.

A request for the first ten samples needs a definition of first. A timestamp alone is insufficient here because forty-two points usually share each timestamp. Include identities to make a stable selection. A small displayed result is useful for inspection; it cannot establish the maximum, minimum, or quality of an entire history.

Tiny example

```
SELECT site_id, point_id, ts, value,
         quality
FROM course.sample
ORDER BY ts, site_id, point_id
LIMIT 5;
```

This returns five rows at `2026-01-05 00:00:00+00` when the client session uses UTC. All five have distinct point identities. Their numeric values can have different units, so do not average this mixed-role excerpt.

Use `SHOW timezone;` to inspect the display setting. We will study timestamp types and windows in week 3.

SQL challenge

Produce a complete equipment inventory ordered by site ID, then label, then equipment ID. Produce the ten most recent raw samples with a deterministic ordering, showing their identifying columns and quality. Save both statements and explain which ordering direction you selected for each column.

Make a third report listing point roles and units in alphabetical order with point identity as the final tie breaker. Keep all forty-two rows for this inventory.

Experiment

Run the tiny example with `LIMIT 1`, then `LIMIT 8`. Observe that a limit changes the number of displayed rows. Reverse only the timestamp direction and predict the earliest timestamp in the result. The fixture's final instant is `2026-01-06 23:55:00+00`. Explain why `LIMIT 5` without an order would not mean latest five.

Acceptance checks

- Your equipment inventory contains three rows in a declared, repeatable order.
- Your latest-sample excerpt contains ten rows from the final fixture instant.

- Your report keeps enough identity columns to distinguish ties and enough context to avoid interpreting mixed units.

Stretch and reflection

Explore `OFFSET 5 LIMIT 5` with the same deterministic ordering. What happens to page boundaries if a new earlier row arrives between page requests? Record the limitation without building an application paginator. Read week 1 references for sorting and limiting behavior.

Previous: Day 3 ·

Day 05 — Count missing values before averaging

Previous: Day 4 · **Week 1 · 45–75 minutes.**
Prerequisites: days 1–4, or equivalent skills.

Goal

Distinguish a missing row from a stored null value, and calculate an average only after stating the role, units, and quality policy.

Before you start

Connect to the course database. Today uses `course.history`, a prepared view that joins sample rows to their point and equipment metadata. One view row still represents one stored sample. Inspect it with `\d course.history` before querying.

Concept

SQL `NULL` represents an unknown or absent value. It is not numeric zero and not an empty string. Use `IS NULL` or `IS NOT NULL`; `value =`

`NULL` does not test missingness. A comparison involving null usually evaluates to unknown, and `WHERE` keeps only true results.

`count(*)` counts rows. `count(value)` counts rows whose value is non-null. `avg(value)` ignores nulls but does not know whether a non-null value has good quality. The course's calculation policy is `quality = 'good' AND value IS NOT NULL`. State that policy explicitly. Missing rows never reach an aggregate; a count needs an expected population before it can reveal their absence.

Tiny example

```
SELECT count(*) AS stored_rows,
       count(value) AS rows_with_value
FROM course.sample;

SELECT count(*) AS
         usable_temperature_rows,
       avg(value) AS mean_deg_c
FROM course.history
WHERE role = 'oat'
  AND quality = 'good'
  AND value IS NOT NULL;
```

The first query returns `24190` stored rows and `24189` non-null values. The second has 1,728 usable outdoor-temperature rows; its mean is

in degrees Celsius. Treat the observed average as a result to record, not a universal outdoor temperature.

SQL challenge

Count the stored null values and list their site, equipment label, role, timestamp, and quality through the history view. Count all `kw` rows and all usable `kw` rows separately. Explain why these counts differ by one. Write an average for one temperature role of your choice, and label its unit in the output alias.

Experiment

Compare `WHERE value IS NULL` with `WHERE value = NULL` on `course.sample`. The first finds one row; the second finds none. Then count `duct_static` rows: 1,726 exist, although three complete five-minute point histories would contain 1,728. Explain why an `IS NULL` search cannot find the two absent pressure rows.

Acceptance checks

- Your report distinguishes 1 stored bad/null sample from 2 entirely absent samples.
- `kw` has 1,728 stored rows and 1,727 usable rows.

- Every average has a named role, unit, and explicit quality filter.

Stretch and reflection

Try `coalesce(value, 0)` only in a display query and explain how replacing unknown measurements with zero could bias an average. When is an explicit unknown label more honest? Consult week 1 references for null semantics and aggregate functions.

Previous: Day 4 ·

Day 06 — Group samples into accountable summaries

Previous: Day 5 · **Week 1 · 45–75 minutes.**
Prerequisites: days 1–5, or equivalent skills.

Goal

Build grouped counts, distinguish `WHERE` from `HAVING`, and check that a summary accounts for every stored sample without combining incompatible quantities.

Before you start

Review yesterday's quality policy and the difference between counting rows and values. A grouped result changes grain. A row that previously meant one observation can become one site total or one role total. Write that new meaning above each saved query.

Concept

`GROUP BY` divides input rows into groups with equal key values. Aggregate functions then produce one result per group. Every selected expression must be a grouping expression, an aggregate, or otherwise allowed by PostgreSQL's grouping rules. Start with explicit grouping columns so the report's meaning stays clear.

`WHERE` filters input rows before grouping. `HAVING` filters completed groups using conditions such as a count threshold. Moving a condition between them can change the question. A count of all stored rows and a count of usable values are both useful, but they are different measures.

Tiny example

```
SELECT role, count(*) AS stored_rows
FROM course.history
GROUP BY role
HAVING count(*) < 1728
ORDER BY role;
```

This returns one role: `duct_static`, with `1726` rows. The threshold comes from three points multiplied by 576 expected instants per point. It is a fixture expectation, not a rule for

arbitrary buildings. This query identifies a short history; it does not identify the two missing instants.

SQL challenge

Create a stored-row count per site and a stored/non-null count per role. Predict the site counts before executing. Add a third report with one row per site and temperature role, containing usable-row count and average in degrees Celsius. Apply the quality filter before grouping. Do not average all roles together.

Save an explanation of the grain of each report. A total across groups should be meaningful: site row counts partition the fixture, while role averages cannot simply be summed into a building temperature.

Experiment

Count rows per quality using all samples. Then apply `WHERE quality = 'good'` and compare. The complete result has 24,189 good rows and one bad row. Use `HAVING count(*) > 1` to see how filtering groups differs from selecting individual good rows. Explain why a group can disappear entirely.

Acceptance checks

- Site totals are 16,126 for `SYN-NORTH` and 8,064 for `SYN-SOUTH`, summing to 24,190.
- The role summary reveals the pressure shortfall and the power null separately.
- Temperature averages use degrees Celsius and exclude unusable values.

Stretch and reflection

Group by both site and quality. Compare the result's grain with grouping by site alone. Which report would help investigate the bad row? Which helps reconcile storage totals? Keep result sets and interpretation together, and read week 1 references for grouping rules.

Previous: Day 5 ·

Day 07 — Week 1 challenge — the first site report

Previous: Day 6 · **Week 1 review · 2–4 hours.** Prerequisites: days 1–6.

Project brief

Produce a small SQL report pack for a colleague who has never seen this synthetic building dataset. Explain the inventory, sample coverage at the count level, and measurement quality using the tools from days 1–6. Work independently: choose the report structure and queries yourself. Allow 2–4 hours, split across sessions if useful.

Use the dedicated course database and preserve the main fixtures. Consult the lab guide for running saved files. The output should be understandable without a live narration. This is a storage and inventory report; counts alone do not establish operating duration or identify every missing timestamp.

Acceptance criteria

- Inventory totals reconcile to 2 sites, 3 equipment identities, and 42 point identities; repeated equipment labels remain distinguishable.
- Stored sample totals reconcile to 24,190, including 1 bad/null observation and the 2-row pressure shortfall against the declared expectation.
- Include a site summary, a role summary, and at least one useful temperature summary with named units and a stated quality policy.
- Every result has a declared grain, descriptive columns, and a deterministic display order where appropriate.
- A bounded detail excerpt supports one finding without printing the entire history.

Deliverables

Submit saved SQL, a compact actual-output transcript, and a short report interpreting three findings. Label results observed or not run. Include one corrected query mistake and explain what made its first answer misleading.

Review questions

What can an equipment label identify reliably? Why do row counts and non-null counts differ? How could a correct aggregate still describe the wrong population? Which questions require joins or explicit time windows you have not learned yet?

Attempt the challenge before reading examples in week 1 references. Record any unmet criterion and a specific next experiment.

Previous: Day 6 ·

Primary reading — Week 1

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 1

- PostgreSQL tutorial
- psql
- SELECT
- Comparison and NULL

- Aggregate functions